

Curingas e Caracteres de Pesquisa no Terminal

Guia Completo de Estudo

Aprenda a usar *, ?, [], {}, **, e outros caracteres especiais para pesquisar arquivos e pastas com eficiência no terminal.

Bash / Zsh

find • grep • ls

Nível: Iniciante → Pro

Conteúdo

- 1 O que são curingas?
- 2 Asterisco * — qualquer sequência
- 3 Interrogação ? — um único caractere
- 4 Colchetes [] — conjuntos e intervalos
- 5 Chaves { } — alternativas explícitas
- 6 Glob duplo ** — recursão de diretórios
- 7 Usando curingas com find
- 8 Curingas com grep (regex)
- 9 Dicas avançadas e combinações
- 10 Tabela de referência rápida

1 O que são curingas?

Curingas (wildcards) são caracteres especiais interpretados pelo shell (Bash, Zsh, Fish...) antes de executar qualquer comando. Eles substituem partes de nomes de arquivos e diretórios, permitindo selecionar múltiplos itens com uma única expressão.

Diferentemente de expressões regulares (regex), os curingas do shell são mais simples e operam especificamente sobre o sistema de arquivos. O processo se chama glob expansion.

■ O shell expande os curingas **ANTES** de passar os argumentos ao comando. Portanto `ls *.txt` é equivalente a `ls a.txt b.txt c.txt` se esses arquivos existirem.

2 Asterisco * — qualquer sequência de caracteres

O asterisco representa zero ou mais caracteres quaisquer (exceto /). É o curinga mais utilizado no dia a dia.

Exemplos com arquivos

```
# Listar todos os arquivos .txt na pasta atual

ls *.txt

# Listar todos os arquivos que começam com 'relatorio'

ls relatorio*

# Listar arquivos que terminam com '_2024.csv'

ls *_2024.csv

# Copiar todos os arquivos .jpg para outra pasta

cp *.jpg /home/user/Imagens/

# Remover todos os arquivos de log

rm *.log

# Mover todos os PDFs para a pasta Documentos

mv *.pdf ~/Documentos/
```

Exemplos com pastas

```
# Listar todas as subpastas de /var/log

ls /var/log/*/

# Acessar qualquer pasta que comece com 'projeto'

ls projeto*/

# Exibir o conteúdo de todos os .conf em /etc

cat /etc/*.conf
```

■ ■ *Atenção: ls * lista também o conteúdo das subpastas! Use ls -d */ para listar apenas diretórios.*

3 Interrogação ? — exatamente um caractere

A interrogação substitui exatamente um caractere qualquer. Útil quando você conhece o tamanho do nome mas não todos os caracteres.

```
# Arquivos com nome de 1 letra + extensão .sh

ls ?.sh

# Arquivos como 'log1.txt', 'log2.txt', 'logA.txt'

ls log?.txt

# Pastas com exatamente 4 caracteres no nome

ls -d ???*/

# Arquivos no formato data: 2024-01-01.csv

ls ???-?-?.csv

# Três interrogações = exatamente 3 caracteres

ls ????.py
```

■ *Use vários ? consecutivos para especificar comprimentos exatos. ?? casa com qualquer par de caracteres (ex: 'ab', '12', 'Z9').*

4 Colchetes [] — conjuntos e intervalos

Os colchetes permitem especificar um conjunto ou intervalo de caracteres aceitos em uma posição. Apenas um caractere do conjunto é casado.

Conjuntos simples

```
# Arquivos que começam com a, b ou c
ls [abc]*

# Arquivos terminados em .jpg ou .png
ls *.[jp][pn]g

# Apenas arquivos que começam com letra maiúscula
ls [A-Z]*

# Arquivos cujo nome começa com dígito
ls [0-9]*
```

Intervalos e classes POSIX

```
# Qualquer letra minúscula na primeira posição
ls [a-z]*

# Negação: NOT — arquivos que NÃO começam com vogal
ls ![aeiouAEIOU]*

# Classe POSIX [:alpha:] — letras a-z e A-Z
ls [[:alpha:]]*

# Classe POSIX [:digit:] — somente dígitos
ls [[:digit:]]*

# Arquivos .c ou .h (comuns em projetos C)
ls *.[ch]
```

Classe	Equivalente	Descrição
[:alpha:]	[a-zA-Z]	Letras maiúsculas e minúsculas
[:digit:]	[0-9]	Dígitos de 0 a 9
[:alnum:]	[a-zA-Z0-9]	Letras e dígitos
[:lower:]	[a-z]	Letras minúsculas
[:upper:]	[A-Z]	Letras maiúsculas
[:space:]	\t\n...	Espaços em branco
[:punct:]	! . , ...	Pontuação

5 Chaves { } — alternativas explícitas

As chaves (brace expansion) expandem uma lista de alternativas separadas por vírgula. Diferente dos outros curingas, as chaves não precisam casar com arquivos existentes — elas apenas geram combinações de texto.

```
# Criar múltiplas pastas de uma vez

mkdir -p projeto/{src,tests,docs,assets}

# Listar arquivos .txt e .md na mesma chamada

ls *.{txt,md}

# Criar backup de um arquivo rapidamente

cp config.yaml config.yaml.{bak,old}

# Extensões múltiplas de imagem

ls foto.{jpg,jpeg,png,webp}

# Renomear mantendo o prefixo (mv com chaves)

mv relatorio.{docx,pdf}

# Sequências numéricas com ..

echo arquivo_{1..5}.txt
```

```
# Saída: arquivo_1.txt arquivo_2.txt ... arquivo_5.txt

# Sequência com passo

echo pag_{01..10..2}

# Saída: pag_01 pag_03 pag_05 pag_07 pag_09
```

■ Chaves podem ser aninhadas: `{a,b{1,2},c}` expande para `a b1 b2 c`. Muito útil para criar estruturas de pastas complexas com um comando.

6 Glob duplo ** — recursão de diretórios

O `**` (globstar) percorre recursivamente todos os subdiretórios. É necessário ativar a opção globstar no Bash com `shopt -s globstar`. No Zsh ele funciona por padrão.

```
# Ativar globstar no Bash

shopt -s globstar

# Listar todos os arquivos .py em qualquer nível

ls **/*.py

# Encontrar todos os arquivos .log recursivamente

ls **/*.log

# Contar todos os arquivos .js do projeto

ls **/*.js | wc -l

# Copiar todos os .sql para uma pasta de backup

cp **/*.sql ~/backup/sql/

# Exibir todos os README.md do repositório

cat **/README.md

# Editar todos os .env com um editor

nano **/.env
```

■ ■ No Zsh use ****** para recursão direta. No Bash, sempre execute **shopt -s globstar** antes, ou adicione essa linha ao **~/.bashrc**.

7 Curingas com find

O comando **find** possui sua própria lógica de padrões via **-name** e **-iname** (case insensitive). Os curingas devem ser colocados entre aspas para evitar que o shell os expanda antes do **find**.

```
# Encontrar todos os .txt a partir da pasta atual
```

```
find . -name "*.txt"
```

```
# Case-insensitive: .TXT .Txt .txt etc.
```

```
find . -iname "*.txt"
```

```
# Somente no diretório raiz (não recursivo)
```

```
find . -maxdepth 1 -name "*.csv"
```

```
# Apenas diretórios com 'backup' no nome
```

```
find / -type d -name "*backup*"
```

```
# Arquivos maiores que 100MB
```

```
find . -name "*.mp4" -size +100M
```

```
# Arquivos modificados nos últimos 7 dias
```

```
find . -name "*.log" -mtime -7
```

```
# Excluir todos os .pyc encontrados
```

```
find . -name "*.pyc" -delete
```

```
# Executar ação em cada arquivo encontrado
```

```
find . -name "*.sh" -exec chmod +x {} \;
```

```
# Combinar dois padrões com -o (OR)
```

```
find . -name "*.jpg" -o -name "*.png"
```

8 Curingas com grep (expressões regulares)

O grep usa expressões regulares, que são mais poderosas que curingas de shell. Abaixo os principais metacaracteres do grep e sua equivalência.

```
# Buscar 'error' em todos os .log da pasta

grep 'error' *.log

# Recursivo: buscar em todos os arquivos de src/

grep -r 'TODO' src/

# -l: apenas exibir nomes dos arquivos que casam

grep -rl 'import os' *.py

# Regex: linhas que começam com 'ERRO'

grep '^ERRO' sistema.log

# Regex: linhas que terminam com ponto e vírgula

grep ';$' *.sql

# . (ponto) casa qualquer caractere único

grep 'arqu.vo' notas.txt

# .* casa qualquer sequência (equivalente ao *)

grep 'inicio.*fim' relatorio.txt

# -E ativa Extended Regex: +, ?, |, {n,m}

grep -E 'erro|falha|critico' app.log

# Case-insensitive

grep -i 'warning' *.txt
```

Glob do shell vs Regex do grep

Função	Glob (shell)	Regex (grep)
Qualquer seq.	<code>*</code>	<code>.*</code>
Um caractere	<code>?</code>	<code>.</code>
Conjunto	<code>[abc]</code>	<code>[abc]</code>
Negação	<code>[!abc]</code>	<code>[^abc]</code>
Início da linha	<code>(não existe)</code>	<code>^</code>
Fim da linha	<code>(não existe)</code>	<code>\$</code>
Zero ou mais	<code>(via *)</code>	<code>x*</code>
Um ou mais	<code>(não existe)</code>	<code>x+ (-E)</code>
Opcional	<code>(não existe)</code>	<code>x? (-E)</code>

9 Dicas avançadas e combinações

Escapar curingas

```
# Use \ para tratar * ou ? como texto literal
```

```
ls arquivo\*.txt
```

```
# Ou use aspas
```

```
ls 'arquivo*.txt'
```

Combinar curingas

```
# Arquivos que começam com [0-9], têm qualquer meio e terminam em .sh
```

```
ls [0-9]*script*.sh
```

```
# Exatamente 8 caracteres de nome antes de .txt
```

```
ls ???????.txt
```

```
# Nome entre 4 e 6 caracteres (combinando ?)
```

```
ls ????.* ??????.* ??????.*
```

Curingas no PATH

```
# Editar qualquer nginx.conf em qualquer subpasta de /etc

nano /etc/*/nginx.conf

# Ver logs de qualquer serviço em /var/log

tail -f /var/log/*/error.log

# Encontrar todos os Makefile do sistema

find /home -name "Makefile"
```

Nomes com espaços

```
# Use aspas para proteger expansão

ls "meus arquivos"/*.txt

# Ou escape o espaço

ls meus\ arquivos/*.txt
```

10 Tabela de Referência Rápida

Curinga	Significado	Exemplo
*	Qualquer sequência de caracteres (0 ou mais)	<code>*.txt</code> → todos os <code>.txt</code>
?	Exatamente um caractere qualquer	<code>arq?.sh</code> → <code>arq1.sh</code> , <code>arqA.sh</code>
[abc]	Um caractere do conjunto a, b ou c	<code>[abc]*.py</code> → <code>a.py</code> , <code>b_util.py</code>
[a-z]	Um caractere no intervalo a até z	<code>[a-z]*.md</code> → <code>leia.md</code>
[!abc]	Qualquer caractere EXCETO a, b, c	<code>[!0-9]*</code> → nomes sem dígito
{a,b}	Expansão de alternativas explícitas	<code>*.{jpg,png}</code> → fotos e pngs
{1..5}	Expansão de sequência numérica	<code>arquivo_{1..5}</code> → 1 ao 5

Curinga	Significado	Exemplo
**	Qualquer nível de subdiretórios (globstar)	<code>**/*.py</code> → todo .py recursivo
^	(grep) Início da linha	<code>grep '^Error' log.txt</code>
\$	(grep) Fim da linha	<code>grep 'OK\$' resultado.txt</code>
.	(grep) Um caractere qualquer	<code>grep 'f.le'</code> → file, fale...
.*	(grep) Qualquer sequência (regex)	<code>grep 'a.*z'</code> → az, abz, abcz

Pratique no terminal! A melhor forma de memorizar curingas é usá-los todos os dias em tarefas reais.